

分类法、本体、知识图谱有什么区别 Taxonomy vs. Ontology vs. Knowledge Graph

核心内容

Q: 这三个词到底分别指什么？ A: 分类法 (taxonomy) 是一棵父子层级树，只回答「这个东西归在哪一类下面」。本体 (ontology) 回答「实体之间是什么关系、按什么逻辑规则推」。知识图谱 (knowledge graph) 是实际建起来的那个东西，里面装着真实的实例数据，而分类法和本体是给它用的组织原则。

Q: 作者用一句什么话把三者的关系摆平？ A: 知识图谱是你建的东西，分类法和本体是图纸。图纸不是要你二选一，而是拿来给知识图谱提供结构和含义。所以问题不是「该选分类法还是本体」，而是「用哪些组织原则来组织我的知识」。

Q: 分类法为什么不够用？ A: 分类法能告诉你「苹果在水果下面、水果在食品下面」，但它说不出实体之间怎么发生关系。「某人下了一个订单、订单里含某个商品」这类事实，父子层级表达不了。要表达关系和推理规则，才需要本体。

Q: 本体一定要用 RDF 三元组实现吗？作者的态度是什么？ A: 不一定，而且作者明确不推荐。RDF 用「主语—谓语—宾语」三元组表达逻辑陈述，表达本体没问题，但描述实例数据的关系与属性时必须拆成大量三元组，复杂度和查询难度都上去了。作者主张用带属性的图模型 (property graph) 装实例数据，本体照样表达得出来。

Q: 知识图谱由哪几部分构成？ A: 四部分——节点 (实体)、关系 (实体之间的连接)、属性 (描述节点或关系的键值)，以及组织原则 (分类法和/或本体)。前三部分装事实，第四部分决定这些事实怎么被组织和解读。

Q: 这件事和 AI agent 有什么关系？ A: 带清晰组织原则的知识图谱，让 agent 取到的上下文是「按含义连起来的」——哪些实体相关、怎么相关、在整个领域里处于什么位置。作者的说法是：agent 可以顺着关系一路走 (从商品走到类别、供应商、订单、工单、相关文档)，而不是只捞回提到了商品名的那几段文本。没有知识图谱，你没法依赖 agent 的推理。

背景补充 / 点评

这是 Neo4j 官方博客的文章，作者 John Stegeman 是 Neo4j 的图数据库产品专家。读的时候要记住这个立场：全文的分析是中立的，而结论落在「用 Neo4j 建知识图谱」上。

文中几个专业名词值得展开：

- SKOS (Simple Knowledge Organization System) 是 W3C 的一个标准，专门用来表达分类法里的「更宽／更窄」关系。传统上分类法就用它建模。
- neosemantics (n10s) 是 Neo4j 的一个插件，作用是在 Neo4j 与 RDF/SKOS/OWL 之间做交换。需要符合 W3C 标准的互操作性时才用得上。
- Schema.org、FIBO、SNOMED CT 是三个现成的共享词表，分别用于网页结构化数据、金融、医疗。文章拿它们举例说明本体的一个用处是**跨系统互操作**——大家用同一套词，数据才能对得上。
- GraphRAG 是「让大模型顺着图的关系取上下文」的那一类做法，与只做向量检索相对。

这篇文章最有价值的一句是那个三层区分：**组织原则和实例数据是两回事**。分类法与本体是「关于这个领域该怎么理解」的知识，实例数据是「这个领域里实际发生了什么」。把两者混在一起，就会出现「每加一个用途就抄一份口径」的局面。这句话本身比文章后半段的产品清单值钱得多。

留白和我不同意的地方有两处。

第一处，文章批评 RDF 三元组「描述实例数据太啰嗦」，这个批评成立，但它举的例子（苹果是水果、香蕉是水果、三文鱼是鱼……）其实是**分类关系**而不是实例数据，用这个例子证明「RDF 不适合装实例数据」不够有力。真正的痛点在于三元组表达属性和多值关系时的膨胀，文章没展开。

第二处更重要：**文章的结论不是从它自己的分析推出来的**。分析说的是「本体是组织原则，实例数据分开存」，而结论跳到了「所以你要建一个知识图谱（用 Neo4j）」。可是「组织原则与实例数据分离」这件事，不需要图数据库也能做到——把本体登记成声明、把实例数据留在已有的数仓或关系库里、由编译器把声明翻译成查询，同样满足那个分离。文章没有讨论这条路，因为讨论它对 Neo4j 没好处。

对我自己的启示。我正在做的统一语义层，按这篇文章的框架看清楚了自己站在哪儿：

- 我的 **tbox（概念）和 rbox（关系）就是本体那一层**——它们回答「课程、考试、学员之间是什么关系」，是组织原则。
- 我的 **指标注册中心是本体的一部分而不是别的东西**：一条指标声明的是「这个业务口径怎么算」，属于「关于这个领域该怎么理解」的知识。
- 而**我没有知识图谱**，我的实例数据留在 StarRocks 与业务库里，由编译器把声明翻译成 SQL。

这个对照让我确认了一件事：**「组织原则与实例数据分离」这条原则我已经拿到了，而拿到它并不需要引入图数据库**。文章那个结论是厂商形状的，不是分析形状的。

同时它也提醒我一个我该警惕的地方：文章说本体的一大用处是**跨系统互操作**（大家用同一套词）。我现在的本体只服务我自己这一条问答链路，还没有第二个消费者。哪天林川那边的 MCE、或者太红那边的 APP 编排要复用同一套口径，「同一套词」这件事才会真正被检验——而那时候暴露的问题，现在这条链路测不出来。

文中金句

"In essence, the knowledge graph is what you build, while taxonomies and ontologies are the blueprints that provide it with structure and meaning." 本质上，知识图谱是你建起来的那个东西，而分类法和本体是给它提供结构与含义的图纸。

"It's not a matter of which one to pick, but rather how to use these principles to organize your knowledge." 问题不是该挑哪一个，而是怎么用这些原则来组织你的知识。

"But a taxonomy doesn't capture how these entities relate to one another." 但分类法表达不出这些实体彼此之间是怎么发生关系的。

"Without a knowledge graph, you can't depend on an agent's reasoning." 没有知识图谱，你没法依赖 agent 的推理。

全文翻译

开篇：为什么要给原始数据加结构

AI cannot inherently understand, reason about, or make decisions based on raw data. To transform raw data into actionable knowledge, you must add structure, relationships, and meaning. If you've been exploring this space, you've likely encountered terms like taxonomy, ontology, and knowledge graph.

AI 本身没有能力理解原始数据、在原始数据上推理，或者据此做决定。要把原始数据变成可用的知识，你必须给它加上结构、关系和含义。在这个领域里摸索过的人，大概都碰到过分类法（taxonomy）、本体（ontology）、知识图谱（knowledge graph）这几个词。

While often used together, they serve different functions. A taxonomy organizes categories as a hierarchy, while an ontology defines the semantic meanings and logical connections between entities. A knowledge graph is the implementation layer that brings these together, storing your instance data while using taxonomies and ontologies as organizing principles.

这几个词经常一起出现，但各自承担的功能不同。分类法把类别组织成一个层级；本体定义实体之间的语义含义与逻辑连接。知识图谱是把这些合到一起的实现层——它存放你的实例数据，同时把分类法和本体当作组织原则来用。

In essence, the knowledge graph is what you build, while taxonomies and ontologies are the blueprints that provide it with structure and meaning. This post unpacks all three terms and provides a mental model for using them to represent knowledge.

本质上，知识图谱是你建起来的那个东西，而分类法和本体是给它提供结构与含义的图纸。这篇文章把三个词逐个拆开讲，并给出一套用它们表达知识的思维模型。

什么是分类法

A taxonomy is one of the simplest organizing principles you can use. It sorts entities into categories and subcategories, giving the graph a hierarchical structure for finding, labeling, and navigating information.

分类法是你能用的最简单的组织原则之一。它把实体分进类别和子类别，给图一个层级结构，便于查找、打标和浏览信息。

Think of an online grocery store. A shopper starts in the Foodstuffs category, then navigates to the Fruit, then to Apple. Each step gets more specific, narrowing down from a broader level. These are represented as a hierarchical parent-child tree in a knowledge graph.

设想一家网上杂货店。购物的人从「食品」这一类进去，然后到「水果」，再到「苹果」。每一步都更具体，从更宽的层级往下收窄。在知识图谱里，这些表示成一棵父子层级树。

Taxonomies are traditionally modeled using the Simple Knowledge Organization System (SKOS) standards, which support broader and narrower relationships. But you can also model taxonomies in a knowledge graph with Entity nodes connected by relationships such as SUBCATEGORY_OF, PARENT_OF, or any label of your choice. You can also import SKOS taxonomies into a knowledge graph through neosemantics (n10s).

传统上分类法用 SKOS (Simple Knowledge Organization System) 标准建模，这套标准支持「更宽」与「更窄」两种关系。不过你也可以在知识图谱里用实体节点加关系来建模分类法，关系名用 `SUBCATEGORY_OF`、`PARENT_OF` 或者任何你自己选的标签。你还可以通过 neosemantics (n10s) 把 SKOS 分类法导入知识图谱。

A taxonomy can tell you that Apple belongs under Fruit, and that Fruit belongs under Foodstuffs. A taxonomy is good for: Consistent labeling and tagging; Faceted search and category-based navigation; Controlled vocabularies that prevent parallel naming drift.

分类法能告诉你苹果归在水果下面、水果归在食品下面。分类法擅长这几件事：一致的打标与标注；分面检索与按类别浏览；用受控词表防止同一个东西出现多套并行叫法。

But a taxonomy doesn't capture how these entities relate to one another. It cannot, by itself, tell you that Apple is a type of Fruit listed under Foodstuffs. When you need to go beyond hierarchy to define how entities relate, that's where an ontology comes in.

但分类法表达不出这些实体彼此之间是怎么发生关系的。光靠分类法本身，说不出「苹果是一种水果，而水果列在食品之下」这件事。当你需要越过层级、去定义实体之间怎么关联时，本体就该上场了。

什么是本体

An ontology describes a domain by providing semantic meanings and logical rules. It provides a model of the important aspects of that domain and how they relate.

本体通过提供语义含义和逻辑规则来描述一个领域。它给出这个领域里重要方面的一个模型，以及这些方面之间如何关联。

Take the same e-commerce example. An ontology goes beyond the broader/narrower relationships of a taxonomy. It defines exactly how entities are related using logical statements: Person placed Order; Order contains Product; Product is a type of Category.

还用同一个电商的例子。本体越过了分类法那种「更宽／更窄」的关系，用逻辑陈述精确定义实体之间怎么关联：某人下了订单；订单包含商品；商品是某个类别的一种。

Ontologies are useful for: Describing semantic meanings that humans or AI systems can understand; Inferencing implicit information through logical deduction; Shared vocabularies for interoperability across systems, such as Schema.org, FIBO in finance, or SNOMED CT in healthcare.

本体的用处有三个：描述人或 AI 系统都能理解的语义含义；通过逻辑演绎把隐含信息推出来；提供跨系统互操作的共享词表，比如 Schema.org、金融领域的 FIBO、医疗领域的 SNOMED CT。

Ontologies are traditionally implemented using the Resource Description Framework (RDF), which uses triples as logical statements: subject-predicate-verb. But this approach is fundamentally challenging to describe the relationships and properties of instance data because it needs to decompose them into multiple triples, increasing complexity and query difficulty.

传统上本体用 RDF（Resource Description Framework）实现，RDF 把逻辑陈述表达成三元组：主语－谓语－宾语。但这个办法在描述实例数据的关系与属性时有个根本困难——它必须把这些东西拆解成多个三元组，复杂度和查询难度都随之上升。

Knowledge graphs with a property graph model offer a more flexible approach to organizing instance data using ontologies.

采用属性图模型的知识图谱，在「用本体来组织实例数据」这件事上提供了更灵活的办法。

什么是知识图谱

A knowledge graph connects real-world entities and their relationships in a property graph model that humans and systems can naturally understand. It has four core parts:

知识图谱在属性图模型里把真实世界的实体与它们之间的关系连起来，而这种表示方式人和系统都能自然地理解。知识图谱有四个核心部分：

Nodes: the entities in the graph, such as Person, Order, or Product. Relationships: the connections between entities, such as TYPE_OF, PLACED_ORDER, or CONTAINS. Properties: the property keys that describe nodes or relationships, such as weight, storage, or price. Organizing principles (taxonomies and/or ontologies): the structures that give the graph meaning, defining how the graph data is organized and interpreted.

节点：图里的实体，比如「人」「订单」「商品」。关系：实体之间的连接，比如 TYPE_OF、PLACED_ORDER、CONTAINS。属性：描述节点或关系的属性键，比如重量、存储条件、价格。组织原则（分类法和／或本体）：给这个图赋予含义的结构，决定图里的数据怎么被组织、怎么被解读。

A knowledge graph can capture taxonomies using nodes and relationships that represent hierarchical structures. It can also capture ontologies using semantic nodes, relationships, and properties to provide meanings and logic. They become the organizing principles that provide meaning and structure to the knowledge graph.

知识图谱可以用表示层级结构的节点与关系来承载分类法；也可以用带语义的节点、关系与属性来承载本体，从而提供含义和逻辑。这两者就成了给知识图谱提供含义与结构的组织原则。

The property graph model can handle many-to-many relationships or multiple relationships of the same type between the same two nodes. It offers much greater flexibility throughout the development process because it isn't limited to the predefined structures of a hierarchical taxonomy or an RDF triple. You can query taxonomies and ontologies alongside the instance data in a single Cypher statement.

属性图模型能处理多对多关系，也能处理同两个节点之间存在多条同类型关系的情况。它在整个开发过程里灵活得多，因为它不受层级分类法或者 RDF 三元组那种预定义结构的限制。你可以在一条 Cypher 语句里，把分类法、本体和实例数据一起查。

For instance, to find out what products a person named Daniel ordered in the Foodstuffs category, you can use a single query: `MATCH (d:Person {name: "Daniel"})-[:PLACED]->(o:Order)-[:CONTAINS]->(p:Product)-[:TYPE_OF]->(t:Type)-[:TYPE_OF]->(c:Category {name: "Foodstuffs"}) RETURN d, o, p, t, c`

举个例子，想知道名叫 Daniel 的人在「食品」这一类下订过哪些商品，一条查询就够（见上方 Cypher 语句）。

This property graph model also makes it simple to query shared attributes, such as finding other persons with the same address as Daniel and what products they ordered.

属性图模型也让「查共享属性」变得简单，比如找出与 Daniel 地址相同的其他人，以及那些人订过什么商品。

怎么用这三者表达知识

As standalone models, a taxonomy helps you represent hierarchies, and an ontology helps you represent meanings between entities. It's not a matter of which one to pick, but rather how to use these principles to organize your knowledge.

作为各自独立的模型，分类法帮你表达层级，本体帮你表达实体之间的含义。问题不是该挑哪一个，而是怎么用这些原则来组织你的知识。

The best way to represent knowledge is to build a knowledge graph and use taxonomy and/or ontology to provide structure and meaning to it. This approach gives you a complete knowledge system that's easy to search for, understand, and reason about.

表达知识的最好办法是建一个知识图谱，然后用分类法和／或本体给它提供结构与含义。这个做法给你一套完整的知识系统，便于检索、理解和推理。

Define the use case and questions you need answers for. Define your graph schema. Think of your instance data as nodes, relationships, and properties in the knowledge graph. Then consider how to best organize them.

第一步，把使用场景和你需要答案的那些问题定下来。第二步，定义你的图模式：把实例数据想成知识图谱里的节点、关系和属性，然后考虑怎么组织最好。

Use taxonomy if your data contains categories or hierarchies. Model node labels to represent categories. Model relationships between categories to represent hierarchies. Use ontology if your data needs meaning or logical rules. Model relationships to represent logical connections and relationship labels to provide semantic context.

如果你的数据里有类别或层级，用分类法：把节点标签建模成类别，把类别之间的关系建模成层级。如果你的数据需要含义或逻辑规则，用本体：把关系建模成逻辑连接，用关系的标签提供语义上下文。

Ingest your instance data into the knowledge graph according to the graph schema. Adjust the graph schema as your use cases evolve.

按图模式把实例数据灌进知识图谱。随着使用场景演进，再调整图模式。

三者对照表

（原文给了一张三列对照表，逐行译出如下）

它是什么：分类法是父子关系构成的层级树；本体是实体之间的语义含义与逻辑规则；知识图谱是由节点、关系、属性以及组织原则（分类法和／或本体）构成的图。

数据模型：分类法用带标签的属性图，或者 SKOS；本体用带标签的属性图，或者 RDF；知识图谱用带标签的属性图。

查询语言：分类法在图数据库里用 Cypher 或 GQL，在 SKOS 里用 SPARQL；本体经 neosemantics (n10s) 导入后用 Cypher 或 GQL，在 RDF 系统里用 SPARQL；知识图谱用 Cypher 或 GQL。

用途：分类法用于分类与打标；本体用于互操作与推理；知识图谱用于图遍历、多跳推理、GraphRAG 和 AI 记忆。

怎么落地：分类法是把节点归进父子关系；本体是定义节点之间的含义与逻辑连接；知识图谱是按组织原则设计图模式，然后灌入实例数据。

知识图谱怎么让 AI agent 推得更好

A knowledge graph with a clear taxonomy or ontology lets AI agents retrieve context that is connected by meaning: which entities are related, how they are related, and where they sit in the broader domain.

一个带清晰分类法或本体的知识图谱，让 AI agent 取回的上下文是按含义连起来的：哪些实体相关、怎么相关、在整个领域里处于什么位置。

AI agents retrieve this connected data and perform multi-step reasoning through GraphRAG. Instead of doing a simple vector search that uses a collection of disconnected text chunks, they can traverse relationships throughout the graph.

AI agent 通过 GraphRAG 取回这些连起来的数据，并执行多步推理。不再是做一次简单的向量检索、拿回一堆互不相连的文本片段，而是可以顺着关系在整张图里走。

For example, an AI agent answering a product-impact question can traverse from a product to its category, supplier, orders, support tickets, and related documents, rather than retrieving only the chunks that mention the product name. This makes the AI agent's answers more accurate and explainable. Without a knowledge graph, you can't depend on an agent's reasoning.

举个例子，一个 agent 在回答「某个商品受什么影响」时，可以从这个商品一路走到它的类别、供应商、订单、客服工单和相关文档，而不是只捞回提到了商品名的那几段文本。这让 agent 的回答更准确、也更解释得清。没有知识图谱，你没法依赖 agent 的推理。

常见问题

Is a knowledge graph the same as an ontology? No. A knowledge graph combines instance data with organizing principles. An ontology is a type of organizing principle.

知识图谱和本体是一回事吗？不是。知识图谱把实例数据和组织原则结合在一起，而本体是组织原则的一种。

Do I need an ontology to build a knowledge graph? No. Many knowledge graphs start simple and add ontology only when formal semantics or interoperability become necessary.

建知识图谱一定要有本体吗？不用。很多知识图谱一开始很简单，只在确实需要形式化语义或互操作性的时候才加本体。

Do I need RDF for ontology? Not necessarily. A knowledge graph can represent ontology directly through labels, relationships, and properties. For W3C-standard interoperability, neosemantics (n10s) supports RDF, SKOS, and OWL exchange.

本体一定要用 RDF 吗？不一定。知识图谱可以直接用标签、关系和属性来表示本体。需要符合 W3C 标准的互操作性时，neosemantics (n10s) 支持 RDF、SKOS、OWL 的交换。

What's the difference between a taxonomy and an ontology? A taxonomy classifies things into a hierarchy. An ontology defines a domain, providing semantic meanings and logical rules.

分类法和本体有什么区别？分类法把东西分进一个层级。本体定义一个领域，提供语义含义与逻辑规则。

What comes first — taxonomy, ontology, or knowledge graph? There is no required order. The knowledge graph is what you build. Taxonomy and ontology are organizing principles you use to organize your knowledge graph.

分类法、本体、知识图谱，哪个先来？没有规定的顺序。知识图谱是你建的那个东西；分类法和本体是你用来组织这个知识图谱的组织原则。

Where does GraphRAG fit? AI agents traverse knowledge graphs via GraphRAG to retrieve connected context and perform multi-step reasoning, producing accurate, explainable outputs.

GraphRAG 处在什么位置？AI agent 通过 GraphRAG 遍历知识图谱，取回连起来的上下文并执行多步推理，产出准确且解释得清的结果。

参考链接

- [Simple Knowledge Organization System \(SKOS\)](#) — W3C 的分类法建模标准，用「更宽／更窄」关系表达层级
- [neosemantics \(n10s\)](#) — Neo4j 插件，在属性图与 RDF／SKOS／OWL 之间做交换
- [Ontologies in Neo4j: Semantics and Knowledge Graphs](#) — 同站更深入讲本体在 Neo4j 里怎么落地的文章
- [Schema.org](#) — 网页结构化数据的共享词表，本文举的互操作例子之一
- [FIBO](#) — 金融业务本体，金融领域的共享词表
- [SNOMED CT](#) — 医疗术语标准，医疗领域的共享词表
- [RDF vs. Property Graphs](#) — 本文「RDF 描述实例数据太啰嗦」那个论点的出处
- [What is a knowledge graph?](#) — 知识图谱的定义篇
- [Cypher 文档](#) — 文中查询语句用的图查询语言

- [Cypher 与 GQL](#) — GQL 是图查询的 ISO 标准，与 Cypher 的关系
- [多跳推理](#) — 知识图谱加大模型做多跳推理
- [什么是 GraphRAG](#) — 顺着图取上下文的那类检索做法
- [GraphRAG 让 agent 真实性提高 80%（独立研究）](#) — 文章开头横幅引的那份报告